

TD7: Ingeniería de Datos

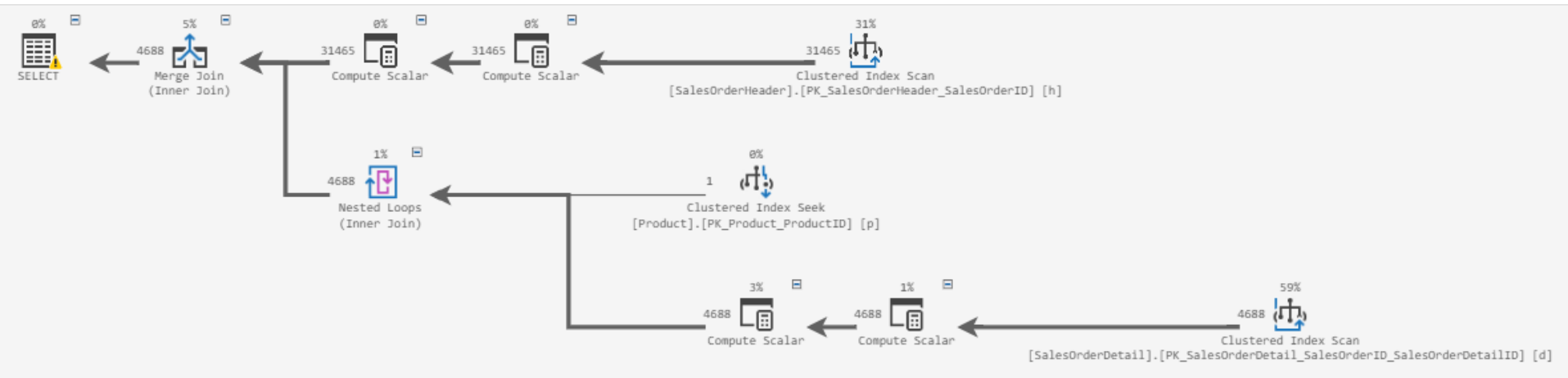
Optimización en SQL Server

Ejecución de consultas

Ejecución de consultas

-- --

```
select * from
    Sales.SalesOrderDetail d join Sales.SalesOrderHeader h
        on d.SalesOrderID = h.SalesOrderID
    join Production.Product p
        on d.ProductID = p.ProductID
where d.ProductID = 870
```

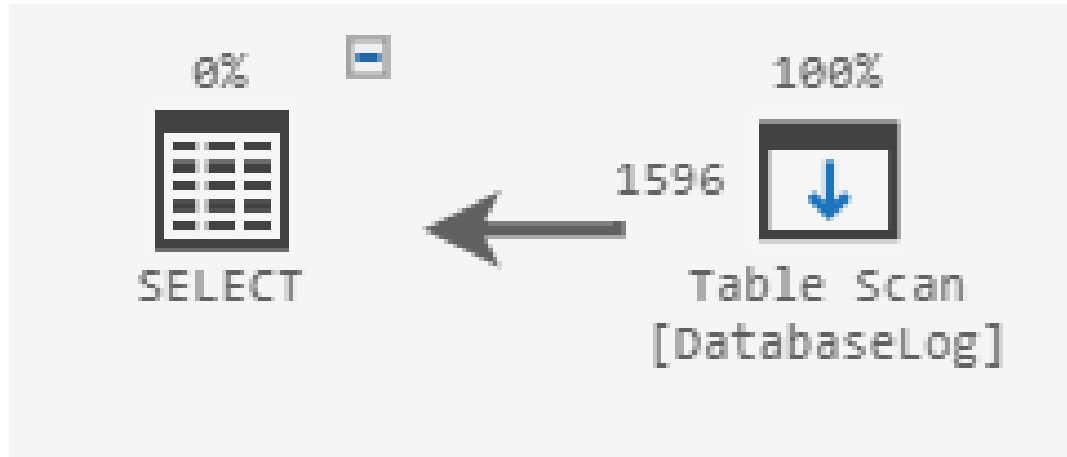


Operadores de acceso a datos

Estructura	Scan	Seek
Heap	 Table Scan	
Clustered Index	 Clustered Index Scan	 Clustered Index Seek
Nonclustered Index	 Index Scan	 Index Seek

Table scan

```
Select * from DatabaseLog
```



Comparando el uso de índices

```
Select * from Person.Address ORDER BY AddressID
```

```
Select AddressID, City, StateProvinceID from Person.Address  
ORDER BY AddressID
```

```
Select AddressID, City, StateProvinceID from Person.Address
```

```
Select AddressID, City, StateProvinceID from Person.Address  
ORDER BY City
```

Operadores de Agregación

SQL Server utiliza dos operadores:

-  Stream aggregate
-  Hash aggregate

Se usan para implementar agregados (SUM, AVG, MAX, etc) y para GROUP BY y DISTINCT.

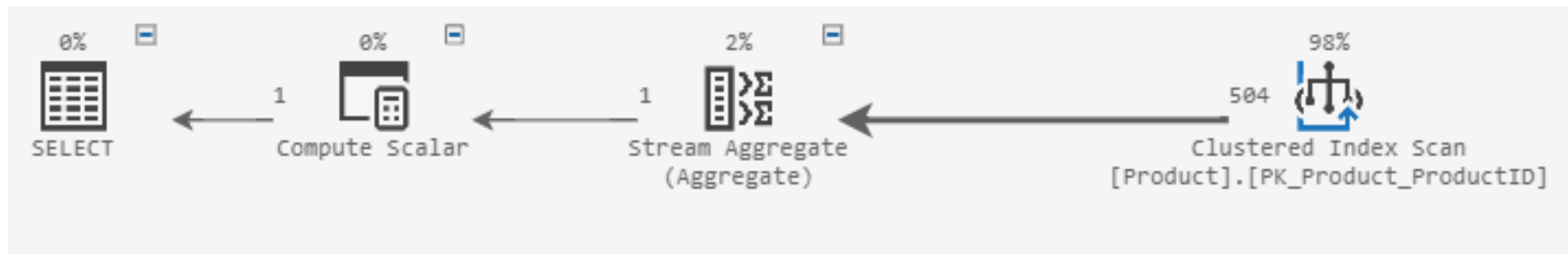
Para algunos operadores se requiere que los datos vengan ordenados, el *Query Optimizer* puede usar un índice existente o puede introducir un *Sort Operator* explícitamente.

Stream aggregate

Consultas que usan un agregado y no tienen una clausula GROUP BY (***scalar aggregates***) siempre se implementan con Stream Aggregate. En caso de usar GROUP BY los datos deben venir ordenados.

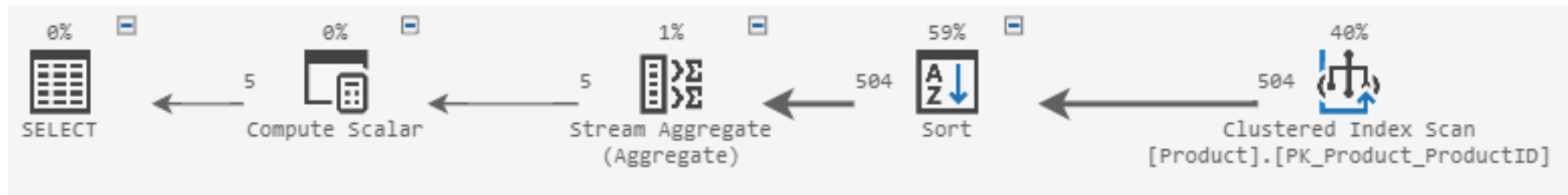
(Las funciones de agregación se pueden aplicar a todas las filas de una tabla, generando un único valor: un **scalar aggregates**. También se pueden aplicar a todas las filas que tienen el mismo valor en una columna o expresión específica (mediante la cláusula group by y, opcionalmente, la cláusula having), generando un valor para cada grupo: un **vector aggregate**).

```
Select AVG( ListPrice ) from Production.Product
```



Stream aggregate

```
Select ProductLine, COUNT(*) from Production.Product  
GROUP BY ProductLine
```



Stream aggregate

```
Select SalesOrderID, SUM(LineTotal) from Sales.SalesOrderDetail  
GROUP BY SalesOrderID
```



Hash aggregate

Hash Match

El Hash Aggregate cómo el Hash Join se implementa con el operador físico **Hash Match**

```
Select City, COUNT(City) from Person.Address  
GROUP BY City
```

El optimizador de consultas puede seleccionar un *Hash Aggregate* para tablas grandes donde los datos no están ordenados, *no es necesario* ordenarlos y su cardinalidad se estima en solo unos pocos grupos.



Hash vs Stream aggregate

- *Hash Aggregate* ayuda cuando los datos no están ordenados. Si se crea un índice que pueda proporcionar datos ordenados, entonces el optimizador de consultas puede seleccionar un *Stream Aggregate*.
- Si la entrada no está ordenada pero se pide el orden explícitamente en una consulta, el optimizador de consultas puede introducir un *Sort operator* y un *Stream Aggregate* o puede decidir usar un *Hash Aggregate* y luego ordenar los resultados

Distinct Sort

Una consulta que usa la palabra clave **DISTINCT** puede ser implementada por *Stream Aggregate*, *Hash Aggregate* o por un operador *Distinct Sort*.

```
Select DISTINCT(JobTitle) from humanResources.Employee
```



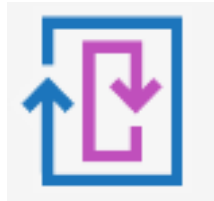
Juntas

Juntas

El optimizador de consultas debe tomar dos decisiones importantes con respecto a las juntas:

- la selección de un orden de las juntas
- la selección del algoritmo de junta

Nested Loops Join



```
Select * from humanResources.Employee as e  
INNER JOIN Sales.SalesPerson as s  
on e.BusinessEntityID = s.BusinessEntityID
```



Es muy efectivo si el *outer input* es suficientemente pequeña y el *inner input* es grande pero indexada.

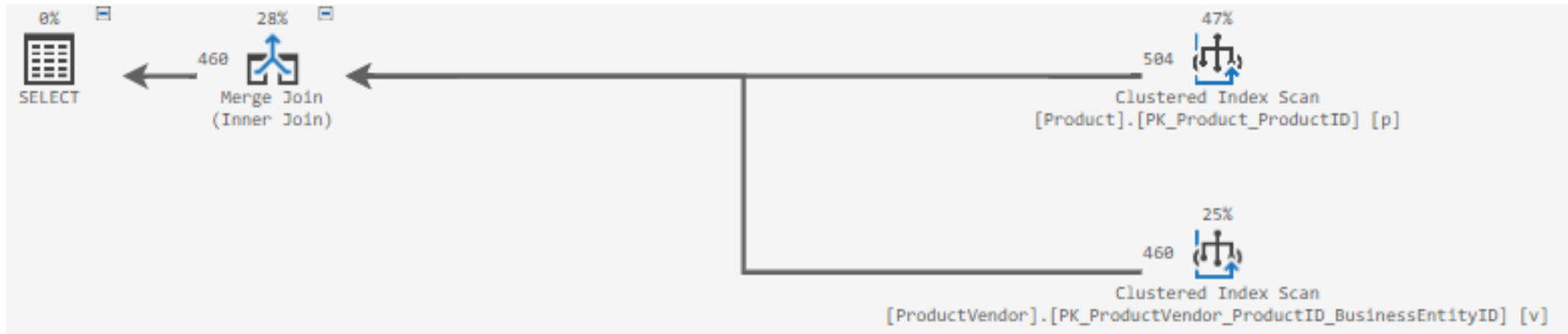
Merge Join y Hash Join requieren que la junta tenga al menos un predicado de junta basado sobre un operador de igualdad (ej. equi join)

Merge Join



Select

```
p.ProductID, v.BusinessEntityID
from Production.Product as p
  INNER JOIN Purchasing.ProductVendor as v
on p.ProductID = v.ProductID
```

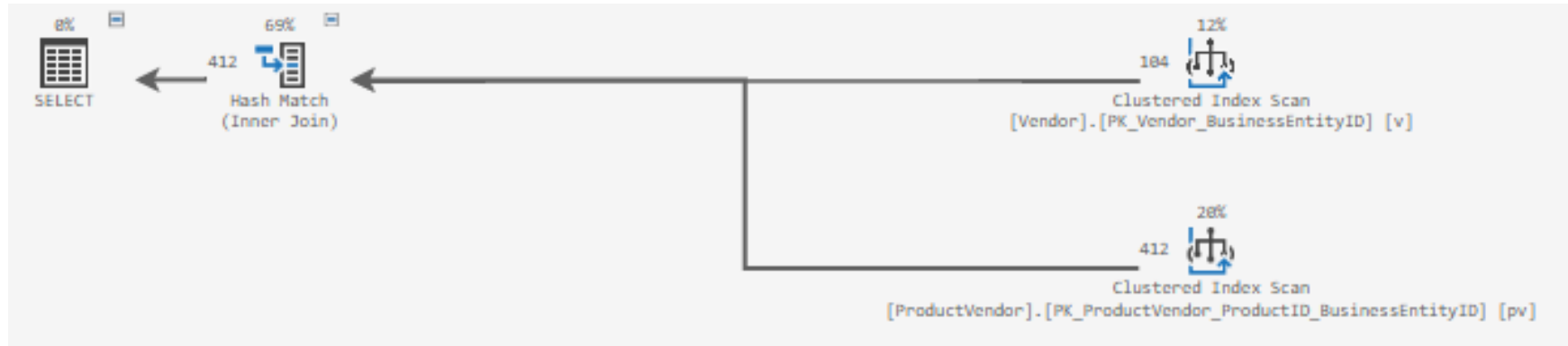


Hash Join



Select

```
    pv.ProductID, v.BusinessEntityID, v.Name
from Purchasing.ProductVendor as pv
    INNER JOIN Purchasing.Vendor as v
on pv.BusinessEntityID = v.BusinessEntityID
WHERE StandardPrice > 10
```

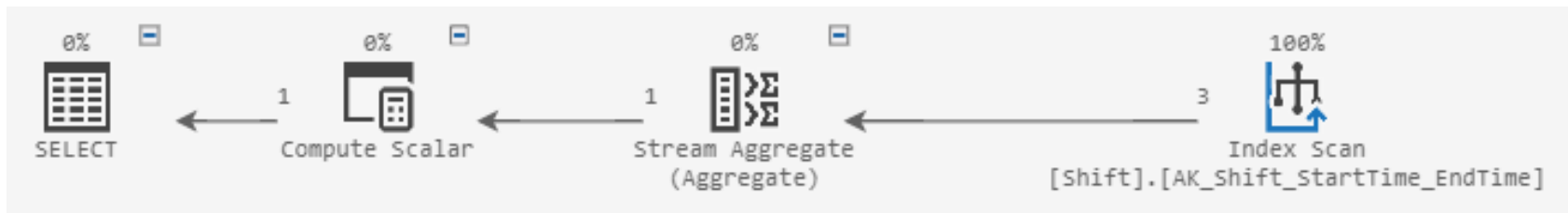


Juntas - Resumen

Join Type	Index on Joining Columns	Usual Size of Joining Tables	Presorted	Join Clause
Hash	Inner table: Not indexed Outer table: Optional Optimal condition: Small outer table, large inner table	Any	No	Equi-join
Merge	Both tables: Must Optimal condition: Clustered or covering index on both	Large	Yes	Equi-join
Nested loop	Inner table: Must Outer table: Preferable	Small	Optional	All

Compute Scalar

Select **count**(*) from HumanResources.Shift



Select **COUNT_BIG**(*) cRows from HumanResources.Shift



Filter



```
SELECT City , COUNT ( City ) AS CityCount
      FROM Person.Address
GROUP BY City
HAVING COUNT ( City ) > 1
```

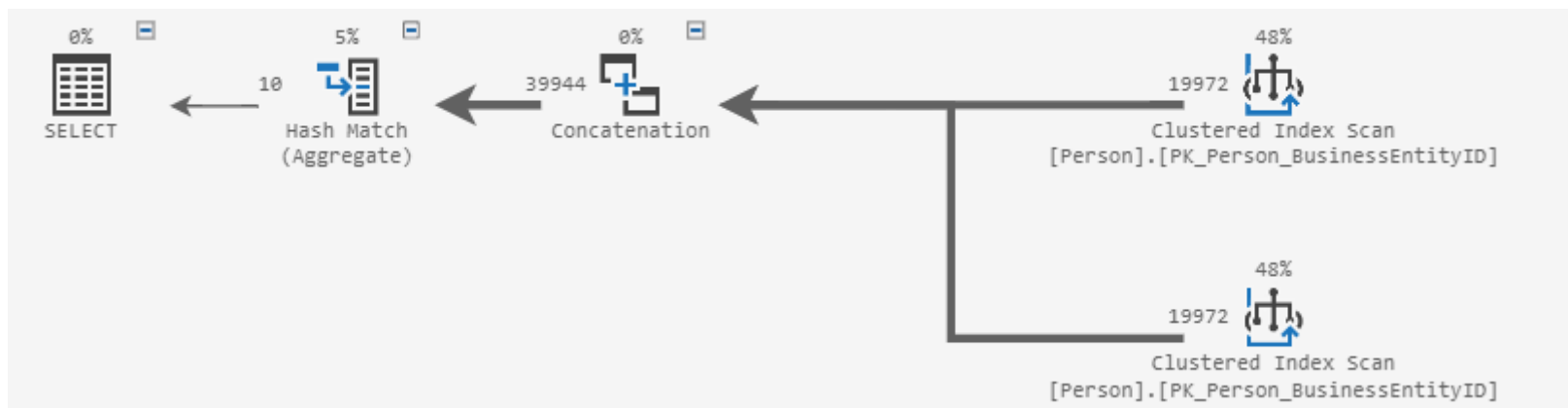


Union

OPERADORES

Hay varios operadores que el optimizador utiliza para resolver la unión: merge, hash, concat

```
select Suffix from Person . Person  
      UNION  
select Suffix from Person . Person
```



¿Que es más costoso UNION o UNION ALL?

Integridad referencial

```
SELECT a.AddressID , a.StateProvinceID
      FROM Person.Address AS a
      JOIN Person.StateProvince AS sp
        ON a.StateProvinceID = sp.StateProvinceID
 WHERE a.AddressID = 27234;
```

```
SELECT *
      FROM Person.Address AS a
      JOIN Person.StateProvince AS sp
        ON a.StateProvinceID = sp.StateProvinceID
 WHERE a.AddressID = 27234;
```



Bibliografía

— — —

 **[DSC19] “Database System Concepts”, A. Silberschatz, H.**

Korth, S. Sudarshan, 2019

📌 Procesamiento de consultas: Capítulo 15

📌 Optimización: Capítulo 16, 16.1 a 16.4

 **“SQL Server Execution Plans”, G. Fritchey, 2018**